Lab 6: FSM Implementation

Dylan Catte, Will Inkrott, Vova Herdman

FSM State Transition Graph

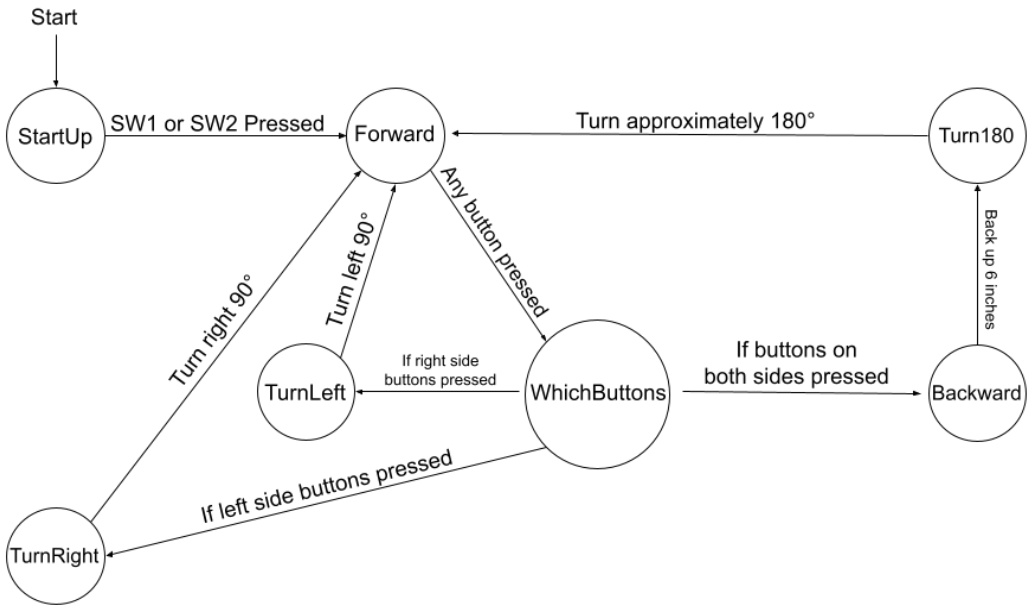


Figure 1: FSM State Transition Graph

bump.c

```
C/C++
// Negative logic bump sensors
// P4.7 Bump5, left side of robot
// P4.6 Bump4
// P4.5 Bump3
// P4.3 Bump2
// P4.2 Bump1
// P4.0 Bump0, right side of robot

#include <stdint.h>
#include "msp.h"
#include "..\inc\Motor.h"
#include "..\inc\Clock.h"
#include "bump.h"

/**
 * Initialize bump sensors
 */
void Bump_Init(){
```

```
    P4 -> SEL0 &= ~0b11101101; // Init GPIO
    P4 -> SEL1 &= ~0b11101101;
    P4 -> DIR &= ~0b11101101;  // Set input
    P4 -> REN |= 0b11101101;   // Enable resistor
    P4 -> OUT |= 0b11101101;   // Make pull up resistor
}


/**
 * Read current state of 6 switches
 * returns 2 for turning left
 * returns 3 for turning right
 * returns 5 if going backwards
 * returns 1 if none are pressed
 */
uint8_t WhichButton() {
    // Align bits
    uint8_t right = P4 -> IN & 0b00000001 | (P4 -> IN & 0b00000100 | P4 -> IN &
0b00001000) >> 1;
    uint8_t left = ((P4 -> IN & 0b00100000) >> 3) | ((P4 -> IN & 0b01000000 |P4 ->
IN & 0b10000000) >> 6);

    if (left < right) {
        // Left bumpers pressed
        return 3;
    } else if (left > right) {
        // Right bumpers pressed
        return 2;
    } else {
        // None are pressed
        if (right == 0b00000111) {
            return 1;
        } else {
            // Start going backwards
            return 5;
        }
    }
}
```

motor.c

```C/C++
#include <stdint.h>
#include "msp.h"
#include "../inc/CortexM.h"
#include "../inc/PWM.h"

// *******Lab 13 solution*******

// ------------Motor_Init------------
// Initialize GPIO pins for output, which will be
// used to control the direction of the motors and
// to enable or disable the drivers.
// The motors are initially stopped, the drivers
// are initially powered down, and the PWM speed
// control is uninitialized.
// Input: none
// Output: none
void Motor_Init(void){
    // Init Motor Direction
    P5 -> SEL0 &= ~0x30;
    P5 -> SEL1 &= ~0x30;
    P5 -> DIR |= 0x30;
    P5 -> OUT &= ~0x30;

    // Init Sleep
    P3 -> SEL0 &= ~0xC0;
    P3 -> SEL1 &= ~0xC0;
    P3 -> DIR |= 0xC0;
    P3 -> OUT &= ~0xC0;

    // Init PWM
    PWM_Init34(15000, 0, 0);
}

// ------------Motor_Stop------------
// Stop the motors, power down the drivers, and
// set the PWM speed control to 0% duty cycle.
// Input: none
// Output: none
void Motor_Stop(void){
    P3 -> OUT &= ~0xC0;
}

// ------------Motor_Forward------------
// Drive the robot forward by running left and
```

```
// right wheels forward with the given duty
// cycles.
// Input: leftDuty  duty cycle of left wheel (0 to 14,998)
//        rightDuty duty cycle of right wheel (0 to 14,998)
// Output: none
// Assumes: Motor_Init() has been called
void Motor_Forward(uint16_t leftDuty, uint16_t rightDuty){
    P5 -> OUT &= ~0x30;
    P3 -> OUT |= 0xC0;
    PWM_Duty3(leftDuty);
    PWM_Duty4(rightDuty);
}

// ------------Motor_Right------------
// Turn the robot to the right by running the
// left wheel forward and the right wheel
// backward with the given duty cycles.
// Input: leftDuty  duty cycle of left wheel (0 to 14,998)
//        rightDuty duty cycle of right wheel (0 to 14,998)
// Output: none
// Assumes: Motor_Init() has been called
void Motor_Right(uint16_t leftDuty, uint16_t rightDuty){
    P5 -> OUT &= ~0x10;
    P5 -> OUT |= 0x20;
    P3 -> OUT |= 0xC0;
    PWM_Duty3(leftDuty);
    PWM_Duty4(rightDuty);
}

// ------------Motor_Left------------
// Turn the robot to the left by running the
// left wheel backward and the right wheel
// forward with the given duty cycles.
// Input: leftDuty  duty cycle of left wheel (0 to 14,998)
//        rightDuty duty cycle of right wheel (0 to 14,998)
// Output: none
// Assumes: Motor_Init() has been called
void Motor_Left(uint16_t leftDuty, uint16_t rightDuty){
    P5 -> OUT |= 0x10;
    P5 -> OUT &= ~0x20;
    P3 -> OUT |= 0xC0;
    PWM_Duty3(leftDuty);
    PWM_Duty4(rightDuty);
}
```

```
// ------------Motor_Backward------------
// Drive the robot backward by running left and
// right wheels backward with the given duty
// cycles.
// Input: leftDuty  duty cycle of left wheel (0 to 14,998)
//        rightDuty duty cycle of right wheel (0 to 14,998)
// Output: none
// Assumes: Motor_Init() has been called
void Motor_Backward(uint16_t leftDuty, uint16_t rightDuty){
    P5 -> OUT |= 0x30;
    P3 -> OUT |= 0xC0;
    PWM_Duty3(leftDuty);
    PWM_Duty4(rightDuty);
}
```

main.c

```
C/C++
#include <start.h>
#include "msp.h"
#include "wait.h"
#include "..\inc\bump.h"
#include "bump.h"
#include "..\inc\Clock.h"
#include "..\inc\LaunchPad.h"
#include "..\inc\Motor.h"

// States for state machine
typedef enum {
    StartUp,
    Forward,
    TurnLeft,
    TurnRight,
    WhichButtons,
    Backward,
    Turn180
} Next;

// Current state of the state machine
Next currentState = StartUp;

int main(void){
  Clock_Init48MHz();  // Set base clock to 48MHz
  LaunchPad_Init();   // Init built-in switches and LEDs
```

```
Bump_Init();        // bump switches
Motor_Init();       // Init motors
initStartButtons(); // Init start buttons

while(1){
    switch (currentState) {
        case StartUp:
            while(startProgram()); // Wait until button is pressed
            currentState = Forward; // Change to Forward state
            break;
        case Forward:
            Motor_Forward(6000,5925); // Start going forward
            Clock_Delay1ms(100);
            currentState = WhichButtons; // Change to WhichButtons state
            break;
        case TurnLeft:
            Motor_Stop();
            Clock_Delay1ms(500);
            Motor_Left(5000, 5000); // Turn left 90 deg
            Clock_Delay1ms(310);
            Motor_Stop();
            Clock_Delay1ms(500);
            currentState = Forward; // Change to Forward state
            break;
        case TurnRight:
            Motor_Stop();
            Clock_Delay1ms(500);
            Motor_Right(5000, 5000); // Turn right 90 deg
            Clock_Delay1ms(310);
            Motor_Stop();
            Clock_Delay1ms(500);
            currentState = Forward; // Change to Forward state
            break;
        case WhichButtons:
            currentState = (Next) WhichButton(); // Determine next state
            break;
        case Backward:
            Motor_Stop();
            Clock_Delay1ms(500);
            Motor_Backward(5000, 5050); // Move backwards for 6 inches
            Clock_Delay1ms(670);
            Motor_Stop();
            Clock_Delay1ms(500);
            currentState = Turn180; // Change to Turn180 state
            break;
```

```
            case Turn180:
                Motor_Stop();
                Clock_Delay1ms(500);
                Motor_Right(5000, 5000); // Turn right 180 deg
                Clock_Delay1ms(620);
                Motor_Stop();
                Clock_Delay1ms(500);
                currentState = Forward; // Change to Forward state
                break;
        }
    }
}
```

Validation Testing Using Debugger

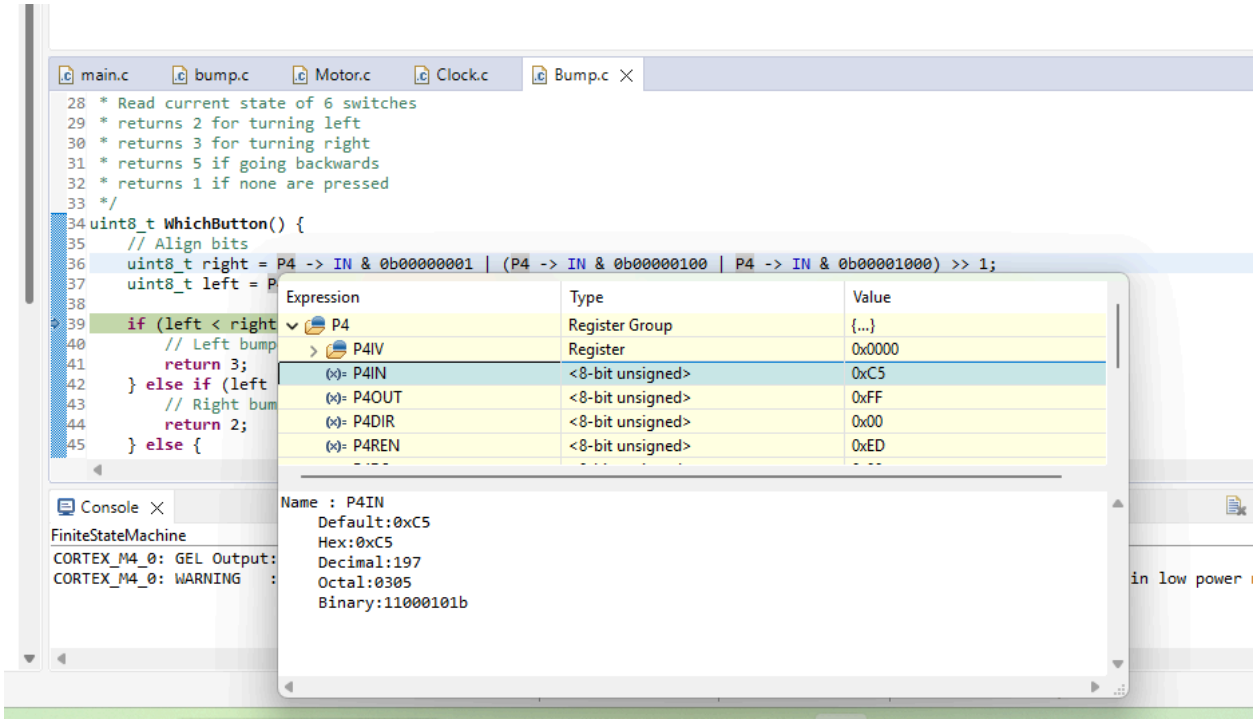


Figure 2: Debugger reading bumper values (left bumper farthest from the center and middle right bumper)

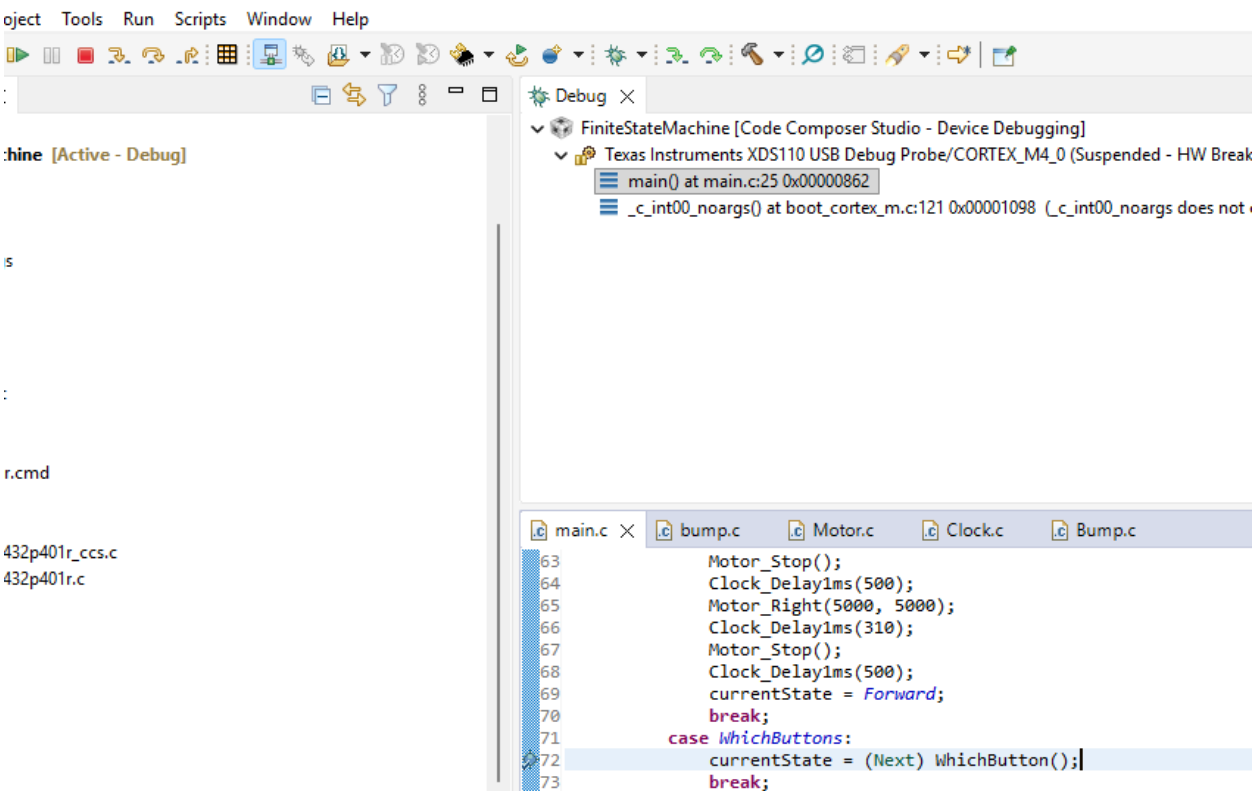


Figure 3: Setting a break point in the debugger

Validation Testing (3 Times, all States)



Figure 4: Robot moving after performing a 90° right turn



Figure 5: Robot moving straight after turning around and performing a 180° turn

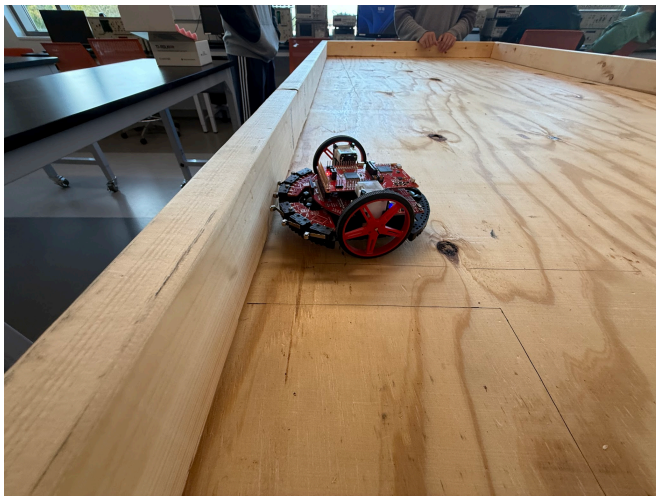


Figure 6: Robot prepares to turn left after a right bumper is pressed

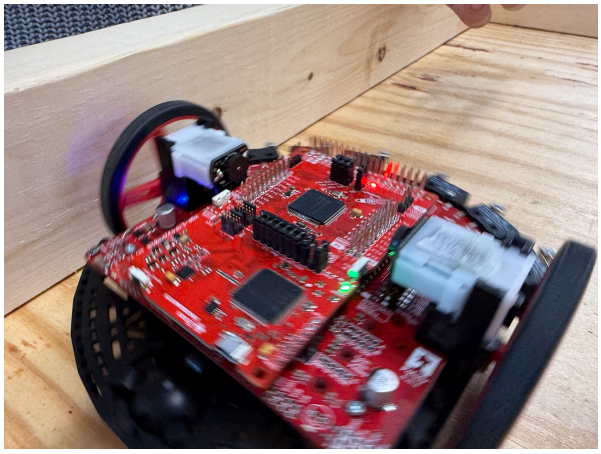


Figure 7: Robot moving after performing a 90° leftturn

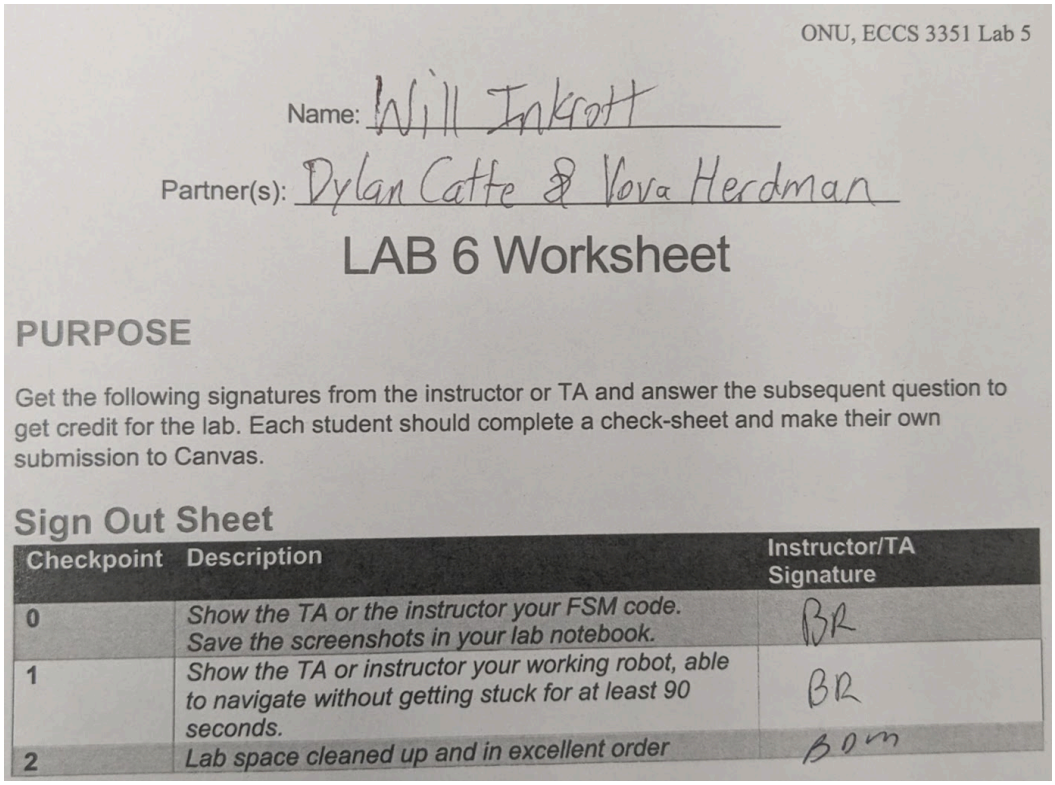
Changes Made as a Result of Validation Testing

When debugging the team found a couple errors. The team forgot to set the next state after the robot completes a state. The team also found that positive logic was used instead of negative logic for the bump switches. The team was able to fix these problems by using the debugger to identify them, and going back in the code to change it. The team also discovered through the debugger that one of the bumpers didn't work, so Doctor McPheron had to help the team replace one of the bump switches.

Test 1 went very well. The only state the team encountered issues with was "Backward". Anytime one button on either side of the robot was pressed simultaneously, the robot would back up 6 inches twice in a row before turning 180°. This was because "Turn180" also called "Backward". The team removed that call and tried again.

Test 2 was better, but the team had an issue with the robot sometimes not moving backwards when bumpers on either side of the robot were pressed simultaneously. To fix this, the team increased the time between polls so when a bumper is hit, there was more time to sense if a second bumper was hit. This was because when two bumpers on either side of the robot were hit simultaneously, sometimes one would be pressed slightly earlier than the other, and the robot would only detect the first press.

Test 3 was supervised by Doctor McPheron. The team had the robot backing up before each left or right turn (because the code did not specify this was not allowed), but Doctor McPheron asked the team to remove the backups prior to the turns, so the team did. After this change the robot worked well and performed every state correctly.



ONU, ECCS 3351 Lab 5

Name: Will Inkrott

Partner(s): Dylan Catte & Vova Herdman

LAB 6 Worksheet

PURPOSE

Get the following signatures from the instructor or TA and answer the subsequent question to get credit for the lab. Each student should complete a check-sheet and make their own submission to Canvas.

Sign Out Sheet

Checkpoint	Description	Instructor/TA Signature
0	*Show the TA or the instructor your FSM code. Save the screenshots in your lab notebook.*	BR
1	*Show the TA or instructor your working robot, able to navigate without getting stuck for at least 90 seconds.*	BR
2	*Lab space cleaned up and in excellent order*	BDM

Figure 8: Lab 6 Sign Out Sheet